Riskwright

An AI-powered credit risk platform

Predicts default probability, explains every decision, derives credit policy rules, and answers plain-English questions about the data.

Home Credit Default Risk | 307,511 applications | 8.07% default rate

NeoStats AI Engineer assignment

Architecture

All business logic in src/. The UI computes nothing.

```
Streamlit UI          no logic, HTTP client only
      |
      v  HTTP
FastAPI               thin wrapper: validates, shapes
      |
      +----------------+----------------+
      v                v                v
 src/ml            src/data       src/talk_to_data
 model, SHAP,      loader,        NL to SQL,
 rules             preprocess     validation, memory
      |                |                |
      +----------------+----------------+
                       v
                 PostgreSQL
      application_train, bureau, previous_application
```

Four services under Compose

postgres, healthchecked

loader, one-shot and idempotent

api, waits on both

ui, waits on api health

Why it matters

Startup races are the usual reason a submission does not run on someone else's machine.

From score to decision

Calibration first, then cost. The second is meaningless without the first.

1. The raw model output is not a probability

Training with scale_pos_weight = 11.39 multiplies the predicted odds by that weight. An average applicant scores near 0.5, not near 0.08.

2. The distortion is a known constant, so it is inverted exactly

$$p\_true = p\_weighted / (p\_weighted + w(1 - p\_weighted))$$

A weighted 0.5 maps back to 1/(1+w) = 0.0807, the measured default rate.

That identity is asserted in the test suite.

3. Only now does a cost threshold mean anything

Catching 64.4% of defaulters instead of 0.2%. Expected cost down 34.9%.

A false negative is a defaulted loan. A false positive is a good customer refused. Assumed 10:1. Threshold chosen to minimise 10*FN + FP.

Threshold	Flagged	Recall	Precision	Expected cost
Naive 0.5	0.0%	0.002	0.659	247,681
Cost-optimal 0.0837	**28.9%**	**0.644**	**0.180**	**161,314**

Risk bands

The two boundaries answer different questions and are set differently.

Boundary	Value	How it is set
t_high Medium / High	0.0837	Cost-optimal. Derived from the 10:1 assumption.
t_low Low / Medium	0.0341	A business judgment. Not an optimum.

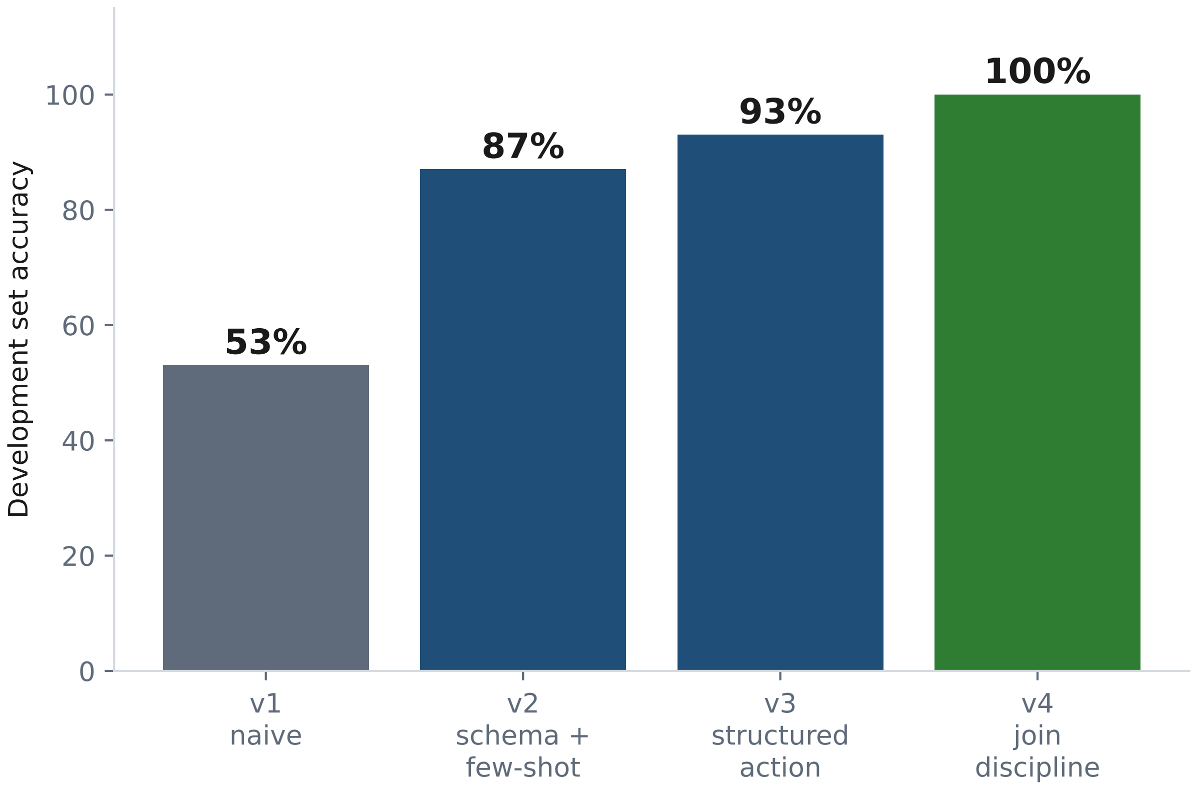
Above t_high the expected cost of approving exceeds the expected cost of refusing. That follows from the cost assumption.

Where automatic approval should stop is a matter of review capacity and risk appetite. Nothing in the data answers it, so t_low is exposed as a policy dial in models/threshold.json rather than presented as a computed result.

Stating which number is derived and which is chosen is the point.

Talk-to-data, measured like a model

30 questions written against the schema before any prompt was tuned.



The development set is a tuning history, not a generalisation estimate.

v4 was tuned on the failures v3 exposed, so its 100% is optimistic by construction.

Held out: 8 further questions, written after the prompt was frozen.

v3	7/8	v4	7/8

88% held out against 100% development. That 12 point gap is the honest number, and v4 shows no reproducible advantage over v3 out of sample.

Refusing is a feature

Two independent mechanisms: a hard gate, and a sanctioned way to decline.

Structured output. One model call returns an action: sql, refuse, or clarify.

A model with no sanctioned way to say 'I cannot' invents a column to fill the silence. Refusal is a first-class result, and it is the cheap path: one call.

Validation gate. Every generated statement is parsed, not string-matched.

```
Rejected   WITH x AS (DELETE FROM application_train RETURNING sk_id_curr)
                SELECT count(*) FROM x

Allowed    SELECT count(*) FROM application_train
                WHERE occupation_type = 'Delete'
```

A DELETE hidden inside a CTE is rejected. The literal string 'Delete' is not. String matching gets both of these wrong. Parsing the tree gets both right.

Single SELECT only, whitelisted tables, every column checked against the live schema, LIMIT forced, 10s timeout, executed as a role holding SELECT and nothing else. Four independent layers, so a parser defect is not by itself an incident.

Fair lending

Found by reading the SHAP output. code_gender was driving credit decisions.

ECOA names sex, marital status and age as protected bases in credit decisions. A model that prices or refuses credit on them is a legal failure, however well it performs. Five attributes are now excluded in the preprocessor, so they cannot reach the model, the SHAP explanation, or the derived rules.

Excluded	Protected basis
code_gender	Sex
name_family_status	Marital status
age_years	Age
cnt_children	Familial status
cnt_fam_members	Familial status

Feature set	ROC-AUC	PR-AUC
With protected	0.7651	0.2491
Without (shipped)	**0.7614**	**0.2427**

Cost of exclusion: 0.0037 ROC-AUC.
Identical split, seed and hyperparameters.

Exclusion is the floor, not the bar. Residual proxies remain and are named:
name_income_type contains 'Maternity leave' and 'Pensioner'; occupation, organization and region columns carry proxy and redlining risk.

Production would additionally need disparate impact testing, proxy detection, adverse action reason codes, ongoing monitoring, and reject inference.

Model

Two models, identical inputs, 5-fold stratified CV.

Model	ROC-AUC	PR-AUC
LightGBM (served)	**0.7614 +/- 0.0026**	**0.2441 +/- 0.0008**
Logistic regression	0.7438 +/- 0.0026	0.2181 +/- 0.0031

The gap is smaller than a headline suggests. Logistic regression comes within 2.4% of a gradient-boosted ensemble, and it is what banks deploy under regulatory pressure because a coefficient per feature is auditable. Worth reporting, not hiding.

LightGBM is served because the requirements reinforce each other: SHAP TreeExplainer is exact on tree models, and rule derivation is naturally a shallow tree.

Imbalance: 8.07% positive, scale_pos_weight 11.39, derived per split and logged. PR-AUC reported alongside ROC-AUC throughout. No SMOTE, no hyperparameter search.

What the data says

Five insights, each computed by the same module the API serves from.

1. A sentinel hides in the employment column. 18.0% of applicants have
days_employed = 365243, about 1,000 years, encoding 'not employed'. Untreated it corrupts every statistic built on the column. Kept as a flag, because it predicts.

2. External credit scores dominate. Lowest quartile of ext_source_3 defaults at
15.1% against 3.5% in the highest. A 4.3x spread from one column.

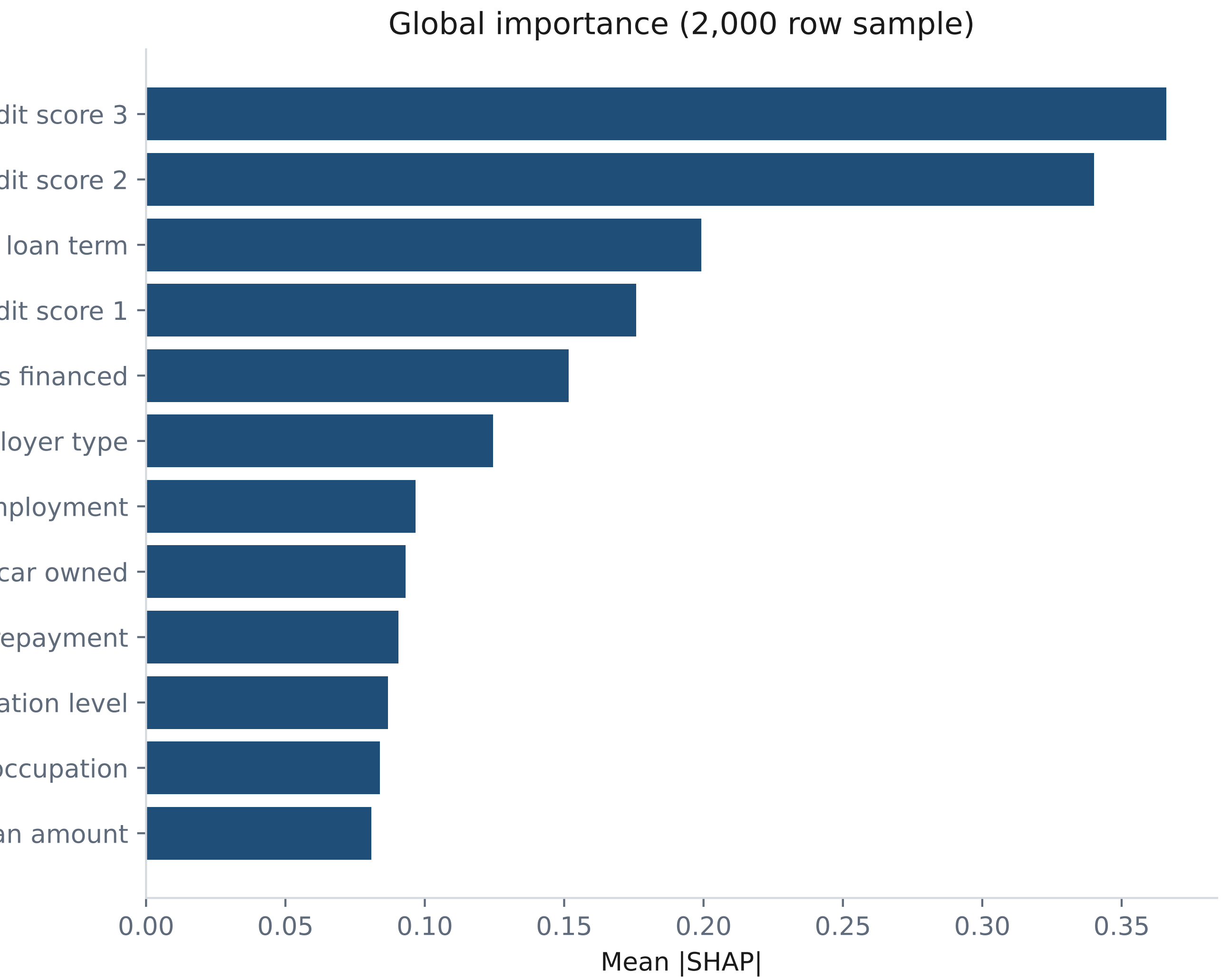
3. Education tracks risk. Lower secondary 10.9%, Academic degree 1.8%.

4. Leverage does not predict default, which is not what you would expect.
Default peaks at 2-4x income (8.77%) and is LOWEST above 6x (7.23%). Likely a selection effect. credit_income_ratio ranks 37th by SHAP; loan term ranks 3rd.
A policy built on a leverage cap would target the wrong quantity.

5. Younger applicants default more. 11.4% under 30, 4.9% at 60 and over.

Explaining one decision

SHAP per prediction, returned as JSON. The UI draws; the API never renders.



Part 4 asks for a non-technical explanation.

A list of signed floats is not one.

Every prediction carries a narrative, generated deterministically from a label map, not by an LLM. It cannot invent a reason the model did not use.

"This applicant has a 44.9% estimated chance of default, which places them in the High risk band. Risk is pushed up mainly by external credit score 3..."

Date columns are negative day offsets, so days_birth = -9461 is shown as 25.9 years, not printed raw.

From model to credit policy

A depth-3 surrogate tree, so the logic fits in a policy document.

Rule	Condition	Support	Default	Lift
R1	external credit score 2 is at most 0.460 and external credit s	5.5%	24.1%	2.98x
R2	external credit score 2 is above 0.460 and external credit sco	4.4%	15.6%	1.93x
R3	external credit score 2 is at most 0.460 and external credit s	15.5%	13.7%	1.70x
R4	external credit score 2 is at most 0.248 and external credit s	3.9%	9.9%	1.23x

Two rule sets. The faithful set agrees with the model on 79.8% of applicants, but keys almost entirely off bureau scores, so it tells a credit officer little they can act on. A second set excludes those scores: 71.4% agreement, and rules on employment length, loan term and goods price that apply to an applicant with no bureau history at all.

Leaf statistics come from counting rows, not from tree_.value. Under class_weight='balanced' that array holds reweighted proportions and reports a 78% default rate on a population whose true rate is 8%. A plausible-looking error.

Engineering

Verified against a running stack, not asserted.

Docker. Four services. Postgres healthchecked; the loader waits for healthy and the API waits for the loader to exit successfully. Verified by cloning this repository into a fresh directory and building with --no-cache.

Security. The chatbot connects as a role holding SELECT on three tables and nothing else. Verified: INSERT, CREATE and DROP are all refused by Postgres. The LLM key is scoped to the API service and never reaches the UI container.

Idempotent load. 740MB in about 150 seconds; a second start skips it in one.

46 tests. The validation gate, the train/serve skew guard, the calibration identity, path resolution, and API shapes.

Token cost. Schema block is 1,656 tokens, not the ~6,000 a full dump would be. Caching is wired but measured not to engage: Haiku 4.5 needs a 4,096 token prefix. Confirmed by padding to 7,393 tokens and watching the cache write, then read.

What this is not

The limitations that would matter to someone deciding to trust it.

Refusal is probabilistic, not guaranteed. The validator makes querying a non-existent column impossible. Declining a question that is semantically unanswerable from columns that do exist is a model judgement, and held-out question H8 was refused in one run and answered wrongly in another.

The model ignores two of the three loaded tables. bureau and previous_application serve the chatbot only. Prior credit history is the largest single improvement available, worth roughly 0.02-0.03 ROC-AUC in published work.

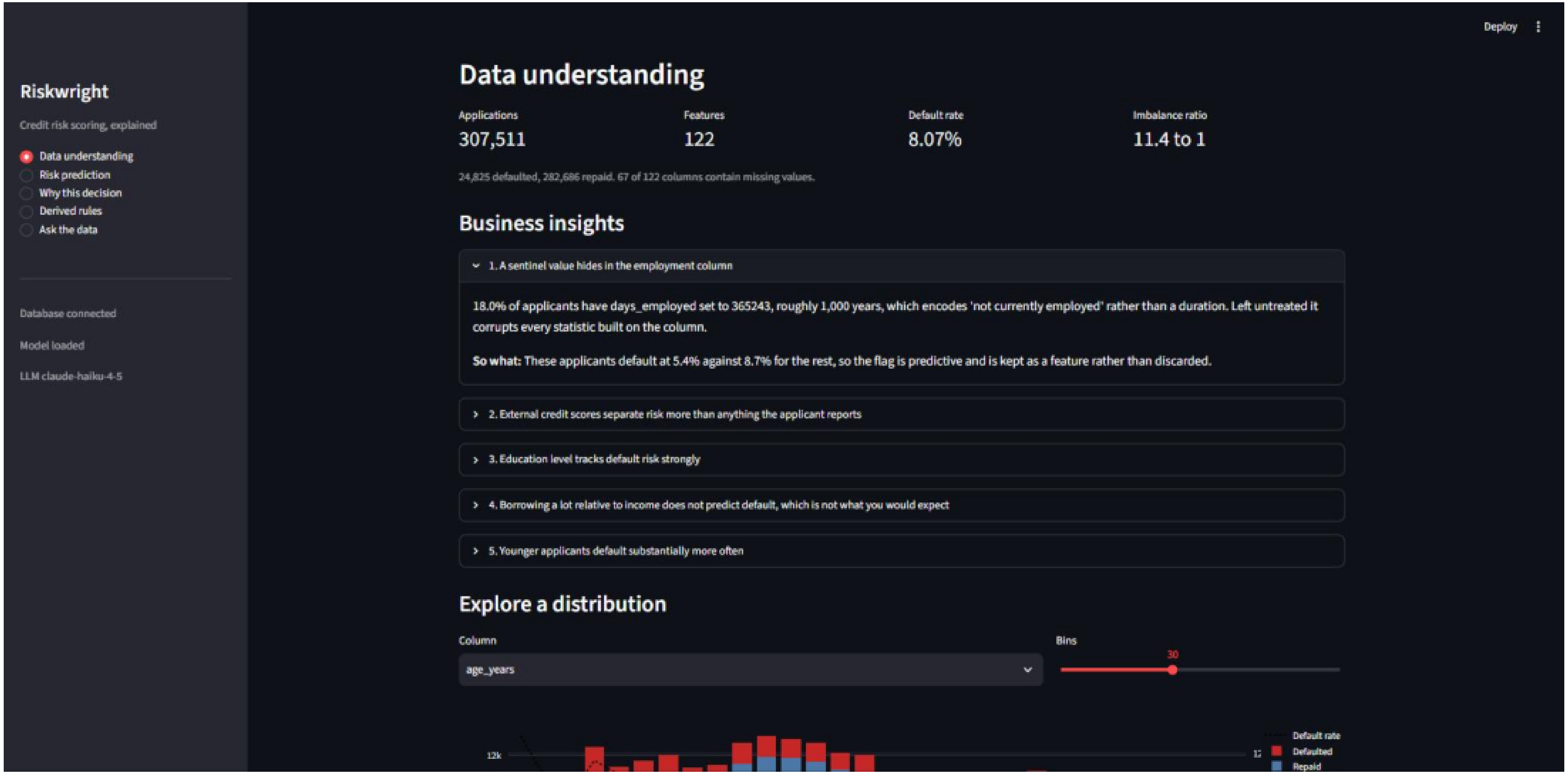
Eight held-out questions is a small sample. A wider set with a proper train and test split is the honest next step.

Fair lending work is incomplete. Excluding protected attributes is the floor. Disparate impact testing, proxy detection and adverse action codes are not built.

Conversation memory is in-process. It does not survive a restart.

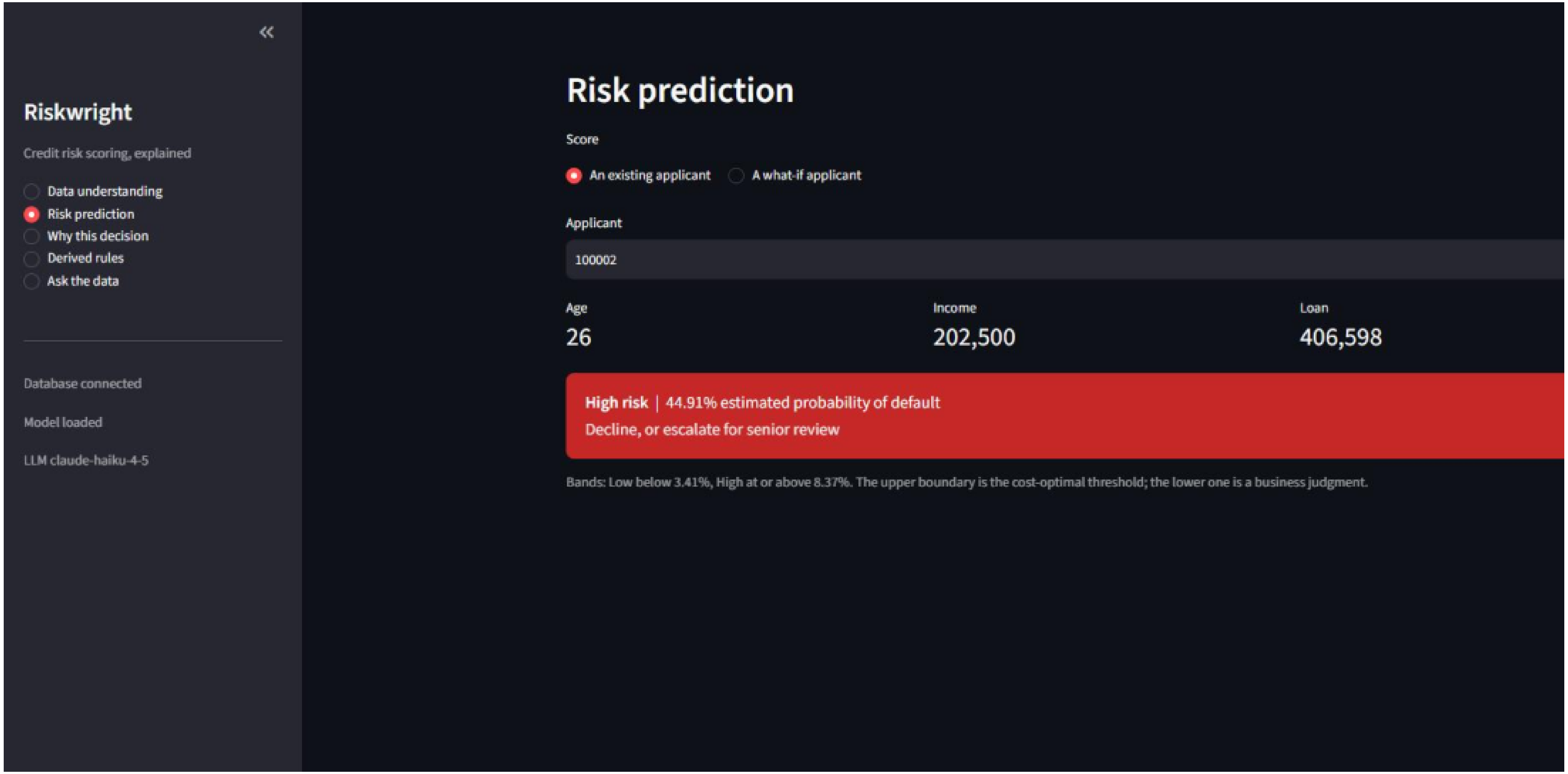
Data understanding

Dataset summary, insights, and the distribution explorer.



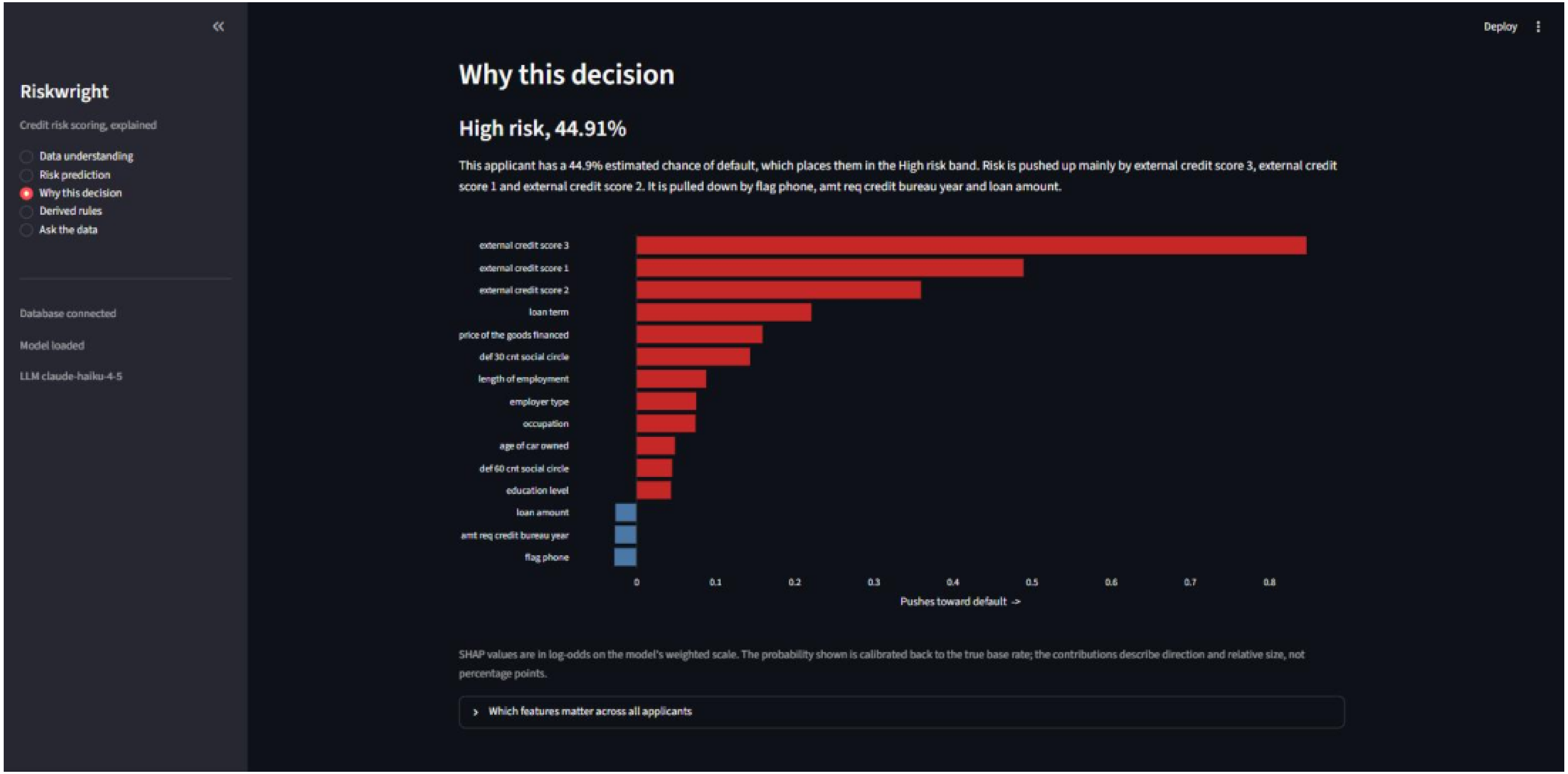
Risk prediction

Score with band and recommended action.



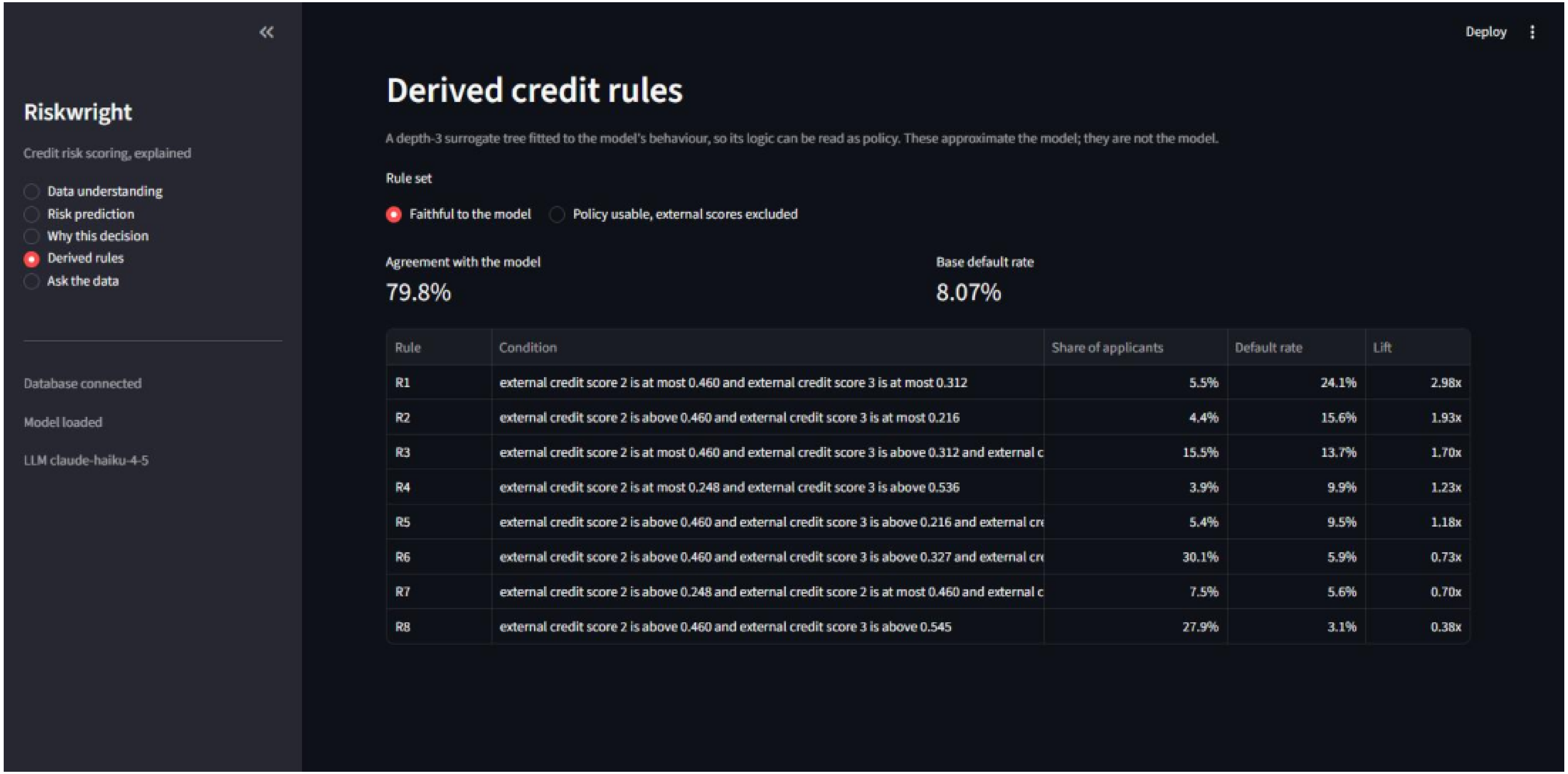
Why this decision

SHAP contributions and the plain-English narrative.



Derived rules

Both rule sets with support, default rate and lift.



Rule	Condition	Share of applicants	Default rate	Lift
R1	external credit score 2 is at most 0.460 and external credit score 3 is at most 0.312	5.5%	24.1%	2.98x
R2	external credit score 2 is above 0.460 and external credit score 3 is at most 0.216	4.4%	15.6%	1.93x
R3	external credit score 2 is at most 0.460 and external credit score 3 is above 0.312 and external c	15.5%	13.7%	1.70x
R4	external credit score 2 is at most 0.248 and external credit score 3 is above 0.536	3.9%	9.9%	1.23x
R5	external credit score 2 is above 0.460 and external credit score 3 is above 0.216 and external cre	5.4%	9.5%	1.18x
R6	external credit score 2 is above 0.460 and external credit score 3 is above 0.327 and external cre	30.1%	5.9%	0.73x
R7	external credit score 2 is above 0.248 and external credit score 2 is at most 0.460 and external c	7.5%	5.6%	0.70x
R8	external credit score 2 is above 0.460 and external credit score 3 is above 0.545	27.9%	3.1%	0.38x

Ask the data

Generated SQL is shown, not hidden.